

Technical Design Document

16/01/2025

“Oil Spill Clean-up”

Authors: Jake Fitzgerald

Project Supervisor: Ross Palmer

Table of contents

1. CRC Cards
2. Architecture
3. Techonologies
4. Reports
 - i. Sprint 1
 - ii. Sprint 2
 - iii. Sprint 3
5. Risks and Challenges
6. Feature Design
7. Art/ Audio/ Animation/ Sound/ UI
8. Summary of Planned Work
9. Videos
10. Testing
11. Audio List

1. CRC Cards

Player	
Responsibilities	Collaborators
• Moving controllable entity • Attacking damagable entities • Taking damage (determines a Game Over state)	• Seal NPC • Interactables • Pickups • Suckable Enemies • Navigation Enemies using A* • Level Tile Map for collision

Projectile	
Responsibilities	Collaborators
• Damaging entities with a Mask Layer listening for a Projectile tag • Spawning particles on death	• Player Projectile • Enemy Projectile • Destructable Walls • All Enemies • Switches • Level Tile Map for collision

Interactables	
Responsibilities	Collaborators
• Allowing for the Player to interact with such as: • Doors - Transition between rooms • Signs - Provide helpful text that displays • Shopkeeper - Similar to Signs that displays text • Chests - Spawns randomised items when opened	• Player

Enemy	
Responsibilities	Collaborators
• Causing damage to the Player character and Seal NPC • Pathfinding using the A* to navigate towards the same Tile the Seal's position is at • Spawning items and particle effects based on death	• Projectile • Player • Score • Seal NPC • A* Grid • Particles • Items

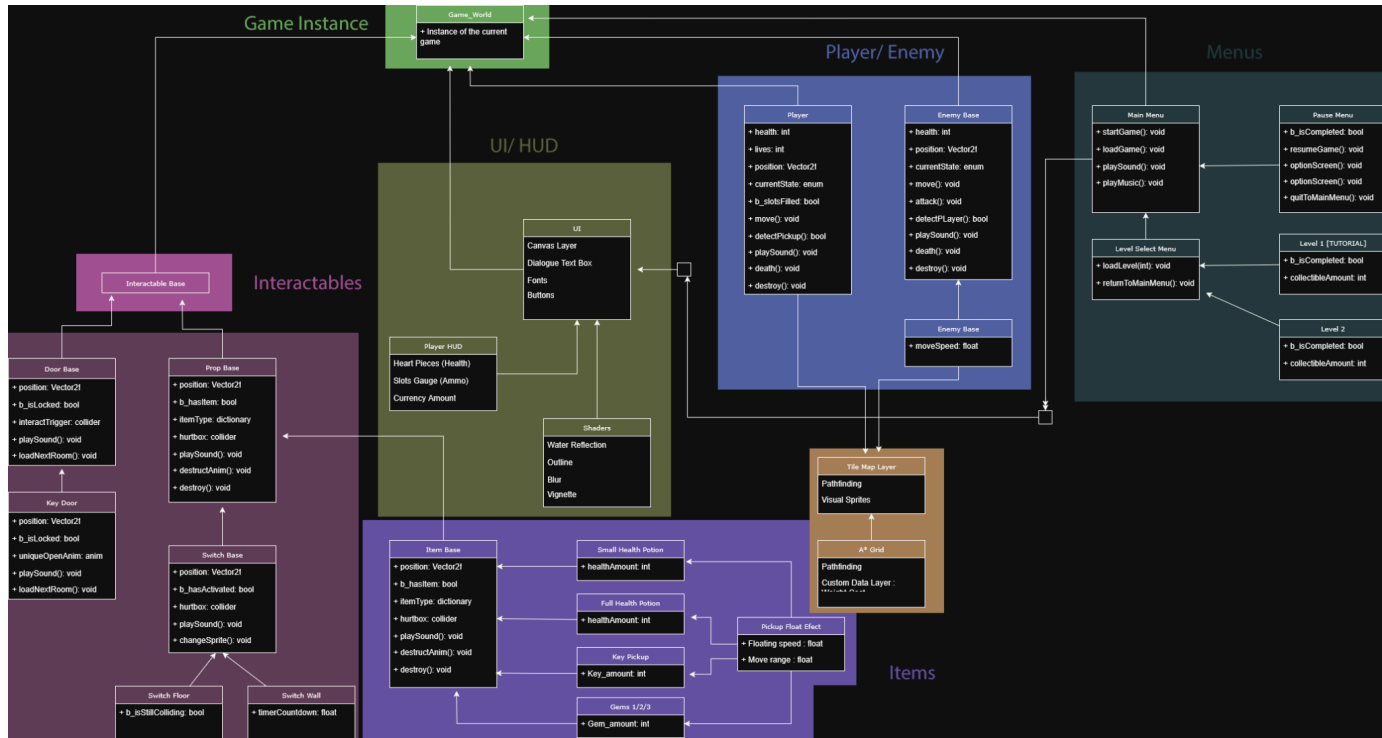
Switches	
Responsibilities	Collaborators
• Activating different triggerable entities within the current map such as a bridge that appears • Can be activated by shooting them with a Projectile (Standing Switch) • Can be activated by standing on them (Floor Switch)	• Projectile • Player

Menus	
Responsibilities	Collaborators
• Buttons that load different scenes (Start Game, Options, Back to Menu)	• Input

Pickups	
Responsibilities	Collaborators
• Giving the Player certain attributes such as recovering their health by a certain amount, adding to their score, filling Key requirement slots, machine gauge slots and their currency amount	• Projectile • Player • Score

2. Architecture

UML Class Diagram



3. Technologies

Research

Using Godot 4.3 for a simple top down game. Have found several resources online to help in both programming and asset creating (sprites, sounds, shaders, etc).

This will be built to run on a web browser, which means we are restricted to certain size limits (1 gigabyte for itch.io). This will not be an issue for code and sprites, however we will need to compress and reduce the amount of audio files used for SFX sounds and music tracks.

Godot supports the extensive use for Tile-Maps which will be used for the creation of the different rooms for each level. *'Tilemap'* is a built-in 2D Node where we can have several layers using what are called *'Tile-Sets'*, this allows us to easily have different layers for specific purposes (animated Tile-Set layer for water sprite animation).

Most collisions will be handled using Collision Shapes 2D with Circle Colliders used for most entities (Player, enemies, props, etc) and Rectangle Colliders for most triggers (Door transitions, traps, interactables, etc).

Installation

Certain plugins and shaders may need to be installed later into the project's development cycle to help with both technical (web based saving) and visual aspects (shaders, temporary art, etc)

Technical Achievement

To be completed at end of project.

4. Reports

Sprint 1 Report

Pre-requisites:

Menu system that links the four different games using four buttons that load a unique URL link. Clicking on a link loads a itch.io page that will play that specific Web based Godot game. Inside of each game's Main Menu, there will be a button that holds a link that when clicked will lead back to the original Menu URL.

Features and Tasks	Time (hours)
--------------------	--------------

Feature 1: Character Movement	3
Task 1: Research for CharacterBody2D	30 mins
Task 2: Player movement on X/Y axis	30 mins
Task 3: Input Handling for directions (Up, Down, Left, Right) [Simple - Not using event functions in Godot]	1
Task 4: Setting up Collision Layers/ Group Tags and testing for Ground, Pickups and Hazards	2
Feature 2: Character Animation	3
Task 1: Player Spritesheet animations corresponding to movement. Each animation created for Move, Idle, Attack, Roll	1
Task 2: Player Animation Tree for blending between each animation transition, aswell as the 4 directions for each animation (Up, Down, Left, Right)	1
Task 3: Code for changing animations based on the current input vector the player is/has moved in	1
Feature 3: Suck Mechanics (Capture)	4
Task 1: Generic actors that have a 'Suck' layer mask	1
Task 2: Having these actors to have a Hurtbox Area2D to allow for a on Area enter and exit triggers, that listen for the Player's Suck collider	1
Task 3: Player's ability for then they are in the "Suck" state to have their Suck Capture hitbox become enabled, adding a single unit to the Slots Gauge, then deleting the entity that was sucked from the scene tree	2
Feature 4: Suck Mechanics (Range pull)	2.5
Task 1: Generic actors that have a 'Suck Range' layer mask	30 mins

Task 2: Having these actors to have a Hurtbox Area2D to allow for a on Area enter and exit triggers, that listen for the Player's Suck Range collider	1
Task 3: Player ability to 'Suck' certain interactable actors by detecting these entities and moving them from their current position towards the Player's current global position	1
Feature 5: Shooting (Simple)	5
Task 1: Projectile scene setup with physics, hitbox, hurtbox Area2D/Collision Shape2D	30 mins
Task 2: Projectile collision/mask layers setup and tested	30 mins
Task 3: Player input to shoot that instantiates the Projectile scene into the current scene tree.	1
Task 4: Return the Player's current facing direction and send it back to Projectile's SetDirection() function	1
Task 5: Projectile movement based on the received Player's direction	1
Feature 6: Slots Gauge System	2
Task 1: HUD UI for representing how many Slots are filled using the Slots_Amount variable in the global Player_Data class	30 mins
Task 2: Shooting limited and integrated with using the Slots system	1
Task 3: Add to Slots_Number_Amount when successfully captured a "Suckable" entity (Enemy, Prop, etc)	30 mins
Feature 7: Scene Manager (Door transitions)	4
Task 1: Create a scene manager script to store all of the scene's paths as strings.	1

Task 2: Create a transition effect when loading the next scene (Visual)	1
Task 3: Have the Player face a certain direction based on which way the door they came from was facing (Up, Down, Left, Right)	2
Feature 8: Basic Enemy (Ground Slime)	5
Task 1: Simple movement (Up, Down, Left, Right) based on a timer	1
Task 2: Animations and Animation Blend Tree setup to work with spritesheet	1
Task 3: Take damage and destroy object on death implementation	2
Task 4: Ability to damage the Player on collision	30 mins
Task 5: Stunned state when being “sucked” by the Player	30 mins
Feature 9: Items (Simple)	2
Task 1: Sprite, Area2D, Collisions and simple code to allow an item to be picked up by the Player	1
Task 2: ‘Health’ Item heals the Player by a specified value that is read by Player_Data	30 mins
Task 3: Instantiate a random Item in the current scene, based on if there are no enemies left within a specific enemy group	30 mins
Task 4: ‘Key’ Item that can be picked up and added to Player_Data. Can be used to unlock a locked Door (disables it’s staticbody2D collider)	1

Work completed:

Features and Tasks	Time Estimate	Time Actual	Complete
--------------------	---------------	-------------	----------

Feature 1: Character Movement	3	6	Y
Task 1: Research for CharacterBody2D	30 mins	1	Y
Task 2: Player movement on X/Y axis	30	1	Y
Task 3: Input Handling for directions (Up, Down, Left, Right)	1	1	Y
Task 4: Setting up Collision Layers/ Group Tags	2	3	Y
Feature 2: Character Animation	3	5	Y
Task 1: Player Spritesheet animations corresponding to movement.	1	1	Y
Task 2: Player Animation Tree for blending between each animation transition	1	2	Y
Task 3: Code for changing animations based on the current input vector the player is/has moved in	1	3	Y
Feature 3: Suck Mechanics (Capture)	4	5	Y
Task 1: Generic actors that have a 'Suck' layer mask	1	1	Y
Task 2: Having these actors to have a Hurtbox Area2D to allow for a on Area enter and exit triggers, that listen for the Player's Suck collider	1	2	Y
Task 3: Player's ability for then they are in the "Suck" state to have their Suck Capture hitbox become enabled, adding a single unit to the Slots Gauge, then deleting the entity that was sucked from the scene tree	2	2	Y
Feature 4: Suck Mechanics (Range	2.5	4	Y

pull)			
Task 1: Generic actors that have a 'Suck Range' layer mask	30 mins	1	Y
Task 2: Having these actors to have a Hurtbox Area2D to allow for a on Area enter and exit triggers, that listen for the Player's Suck Range collider	1	2	Y
Task 3: Player ability to 'Suck' certain interactable actors by detecting these entities and moving them from their current position towards the Player's current global position	1	1	Y
Feature 5: Shooting (Simple)	5	6	Y
Task 1: Projectile scene setup with physics, hitbox, hurtbox Area2D/Collision Shape2D	30 mins	30 mins	Y
Task 2: Projectile collision/mask layers setup and tested	30 mins	30 mins	Y
Task 3: Player input to shoot that instantiates the Projectile scene into the current scene tree.	1	1	Y
Task 4: Return the Player's current facing direction and send it back to Projectile's SetDirection() function	1	2	Y
Task 5: Projectile movement based on the received Player's direction	1	1	Y
Feature 6: Slots Gauge System	2	2	Y
Task 1: HUD UI for representing how many Slots are filled using the Slots_Amount variable in the global Player_Data class	30 mins	30 mins	Y
Task 2: Shooting limited and integrated with using the Slots system	1	1	Y

Task 3: Add to Slots_Number_Amount when successfully captured a “Suckable” entity (Enemy, Prop, etc)	30 mins	30 mins	Y
Feature 7: Scene Manager (Door transitions)	4		
Task 1: Create a scene manager script to store all of the scene’s paths as strings.	1		
Task 2: Create a transition effect when loading the next scene (Visual)	1		
Task 3: Have the Player face a certain direction based on which way the door they came from was facing (Up, Down, Left, Right)	2		
Feature 8: Basic Enemy (Ground Slime)	5	5	Y
Task 1: Simple movement (Up, Down, Left, Right) based on a timer	1	1	Y
Task 2: Animations and Animation Blend Tree setup to work with spritesheet	1	1	Y
Task 3: Take damage and destroy object on death implementation	2	2	Y
Task 4: Ability to damage the Player on collision	30 mins	30 mins	Y
Task 5: Stunned state when being “sucked” by the Player	30 mins	30 mins	Y
Feature 9: Items (Simple)	2	2	Y
Task 1: Sprite, Area2D, Collisions and simple code to allow an item to be picked up by the Player	1	1	Y

Task 2: 'Health' Item heals the Player by a specified value that is read by Player_Data	30 mins	30 mins	Y
Task 3: Instantiate a random Item in the current scene, based on if there are no enemies left within a specific enemy group	30 mins	30 mins	Y
Task 4: 'Key' Item that can be picked up and added to Player_Data. Can be used to unlock a locked Door (disables it's staticbody2D collider)	1	1	Y

Sprint 1 Video link:

https://setuo365-my.sharepoint.com/:v:/g/personal/c00288105_setu_ie/Eb6jcudB6kBGIHD8WVMnLcMBaMitmmMz85P7QTyCyl2yaQ?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJPbmVEcmI2ZUZvckJ1c2luZXNzliwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=9tfVWh&referrer=Outlook.Web&referrerScenario=email-linkwithembed

Sprint 2 Report

Features and Tasks	Time (hours)
Feature 1: Game States	1
Task 1: Research for implementing game states (Lose, Win, Pause) in Godot.	20 mins
Task 2: Creating state machine for each state the game will be in.	10 mins
Task 3: UI for Pause state with different menu buttons (visual only).	30 mins
Feature 2: Loading Screen	1
Task 1: Scene that contains a sprite that covers the screens size and is pure black. Create an animation that turns the	30 mins

transparency of the sprites alpha to 0. Another animation that turns the sprites alpha from 0 back to 255. (Fade in and Fade Out animations).	
Task 2: Loading script that has access to the different games states in the game manager. Also can disable the Players input during the loading duration.	30 mins
Feature 3: Main Menu / Pause Menu	1.5
Task 1: Main Menu system for starting the game, exiting the game and buttons for the user to click on.	30 mins
Task 2: Visual scrolling back sprite, audio for button clicks aswell as background music.	20 mins
Task 3: Pause Menu thats stops gameplay on a button press and resumes gameplay with the same button.	10 mins
Task 4: Has three different buttons: Continue, Options, Exit. Options lets you change the audio mixers (SFX, Music). Exit loads the Main Menu scene and exits from gameplay.	30 mins
Feature 4: Shop Scene	3
Task 1: Shop scene that contains an NPC to interact with that displays a text label with dialogue.	1
Task 2: Purchasable entity (Item) that can be interacted with based on an Area2D trigger collider. Interaction will remove the selected amount from Players current currency amount in Player_Data. This removes the static body that is blocking the Player from collecting the item.	1
Task 3: Shop Manager that holds the values for each Item and how many Items are available to purchase. Labels that display for each Item with the amount of currency needed to purchase that Item.	1
Feature 5: Seal Scene	2

Task 1: Seal end scene creation with tile map, camera limits and boundary collision.	30 mins
Task 2: Seal entity with sprite, animation, Area2D hitbox with sphere hitbox collider. Code for when dealing damage to Seal when colliders with an Enemy.	1
Task 3: Game over state if Seals health is zero. Game win state if Enemy count is zero.	30 mins
Feature 6: Seal Attacker Enemy (Simple)	3
Task 1: Enemy that spawns from offscreen that moves towards the Seals current position in the scene. Has a randomised speed within a range of 2 - 5 pixels per frame.	1
Task 2: Enemy that spawns from offscreen but movement is based on a spline. The end node for the spline always leads to the centre of the screen where the Seals position is.	1
Task 3: Creation for different splines with different patterns and shapes for balance throughout the end scene for defending the Seal.	1
Feature 7: Level Design	1
Task 1: Tutorial layout planned and drawn on paper to showcase each feature in the game.	30 mins
Task 2: Rough blockout for each scene in Godot using a simple grid tile map.	30 mins
Feature 8: Audio Mixers/Option Sliders	1
Task 1: Different Mixer Buses for Master, SFX and Music. Import and assign different audio files for the different buses.	30 mins
Task 2: UI sliders for changing the volume for the different buses in the Options Menu.	30 mins

Feature 9: Audio SFX and Music	1
Task 1: Created AudioStreamer2D for playing music in each scene. Will need to research if needs to be a singleton to play while scene transition.	30 mins
Task 2: Created several AudioStreamer2D for different sounds that play an entities SFX.	30 mins
Feature 10: A Star Pathfinding (Simple)	2
Task 1: Setup a TileMap Layer that draw different sprites based on a Start and Goal cell locations on the grid	30 mins
Task 2: Displays different cells by changing the tile's atlas to a different sprite (Brown to Red) to show the path traveled from the spawn to the target destination.	30 mins
Task 3: Pass a reference for the Player and convert it's X/Y coordinates from Vector2 float into the tile map's int cell format ((0,0) (0,1), (0,2), etc). We do this by using test_a_star.local_to_map(player.global_position)	1 hour
Feature 11: Ranged Enemy	1
Task 1: Facing direction for firing projectiles (can be assigned in the editor when placing the enemy), UP/DOWN/LEFT/RIGHT.	20 mins
Task 2: Instancing a projectile that is inserted into the current scene tree and travels along a set direction. Can damage the player based on tag detection, and then destroy itself.	20 mins
Task 3: SFX sounds for Shooting and Taking Damage. Health int variable to track how many hitpoints, ShootCooldown variable to allow for multiple Ranged Enemys to shoot at different intervals and Set Death(), TakeDamage(), knockback() functions.	20 mins
Feature 12: Particles (Hit/Death/Pickup)	1

Task 1: Animated Sprite that plays a oneshot animated spritesheet. This is a generic particle effect that can be reused.	20 mins
Task 2: Instance the particle inside different entities (Player, Enemy, Pickups, etc), This adds it to the parent node tree (scene), then when animation is finished it is queue freed.	20 mins
Task 3: Create different particles for different effects (Hit, Death, Pickup).	20 mins
Feature 13: Sound Manager	2
Task 1: Autoloaded audio streamer that contains every sound file in the game.	20 mins
Task 2: Mute functionality to turn off either the Music or SFX mixer buses.	20 mins
Task 3: Audio restructuring - remove previous audio streamers from entities and call a "play_X_sound()" function from autoloaded SoundManager instead.	1
Feature 14: Destructible Entity (Wall)	1
Task 1: Area2D with a rectangle collider that listens for a tag with "Projectile". StaticBody2D that blocks Player movement, and is assigned collision layer 2 (Floor/Wall).	20 mins
Task 2: On a collision with Projectile, a timer (0.5 seconds) will start that will queue_free the entity. While that happens a SFX sound plays and a particle effect is added to the scene tree.	20 mins
Task 3: Disable StaticBody2D and the timer will have ended deleting the entity allowing the Player to pass through. Added a flash effect using a similar shader to the Player's flash.	20 mins
Feature 15: Switch (Shoot)	1
Task 1: Area2D with a rectangle collider that listens for a tag with "Projectile". Sprite2D for representing the target you shoot. Custom colours created [Idle = (0,0,0), activated = (1,0,0)].	20 mins

Bool for when the Switch has become active., set to false in Ready() function.	
Task 2: On a collision with Projectile, a timer (0.5 seconds) will start that will queue_free the entity. While that happens a SFX sound plays and a particle effect is added to the scene tree. Active colour assigned when bool is set to true.	20 mins
Task 3: Switch emits a signal that is listened for my another entity in the scene tree (Door, Gate, Bridge, etc).	20 mins
Feature 16: Switch (Ground)	1
Task 1: Area2D with a rectangle collider that listens for a tag with "Player". Sprite2D for representing the target you shoot. Custom colours created [Idle = (0,0,0), activated = (1,0,0)]. Bool for when the Switch has become active., set to false in Ready() function.	20 mins
Task 2: On a collision with Projectile, a timer (0.5 seconds) will start that will queue_free the entity. While that happens a SFX sound plays. Active colour assigned when bool is set to true.	20 mins
Task 3: Switch emits a signal that is listened for my another entity in the scene tree (Door, Gate, Bridge, etc).	20 mins
Feature 17: Chest (Interact)	1
Task 1: Area2D that listens for "Player" tag. When inside a bool 'b_is_active' is set to true and a label displaying a hint text is set to visible. When exited the reverse happens.	20 mins
Task 2: Input function for detecting if the "Interact" key is pressed when inside the trigger area. If it is then a bool 'b_is_opened' is set to true and starts a timer (0.5 seconds) that will call the spawnItems() function. An animation for the chest plays during this time (3 frames lasts for 2 seconds).	20 mins
Task 3: An additional check for if the chest requires a key to open, depending on a bool 'b_reuires_key' which can be set in the editor view. Aswell as a movement function that moves the	20 mins

chest back and forth on the X axis before it being deleted (polish).	
Feature 18: Chest (Item Spawn)	1
Task 1: Preload different item entities that are randomly selected. Create a separate function for each item so that it can be instanced without any errors.	30 mins
Task 2: Items are selected based on how many items a chest can hold (3, 2, 1). Each of these slots then can contain a single item that will be set to spawn at a random position above the chest (Left, Middle, Right). These positions are Node2D in the prefab's scene.	30 mins
Feature 19: Game Over Screen	1
Task 1: Created new scene that contains a simple button to return to the Main Menu scene, so that it will create a new instance of the game and remove all progress made (currency, health amount, score, etc).	20 mins
Task 2: Label that displays "Game Over", SFX sound plays to indicate the new state and a simple background colour rect that shows red to show the Player has failed.	20 mins
Task 3: Game Win Scene is a duplicate of the Game Over scene expect it displays a different label text for "Game Win", a different SFX sound plays and the colour rect is now green to show the Player has won.	20 mins
Feature 19: Boids Flocking algorithm	6
Task 1: Simple character Area2D entity that contains a collider for detecting other entities also tagged as "Boid". A simple sprite to show the Enemy, aswell as a triangular sprite to indicate the direction the entity is facing (faces left by default).	1
Task 2: Create several raycasts that allow the Boid to detect collisions in front of itself. 4 raycasts in total (top, middle upper,	2

middle lower, bottom). This gives the Boid a good vision cone to see in front and above itself. Put these raycasts into an array then loop through them to see if they have collided with any object tagged as "Wall". If it hits a wall we move in the opposite direction (full 180 degrees) based on the raycast.	
Task 3: Area2D trigger collider that detects other colliders tagged as Boid. Is large (50 X 50 pixels) so that it can detect another Boid easily and prevents clipping with another one. When a Boid enters it is appended into an array and this is used in a function called boids().	2
Task 4: Loop through each Boid inside of the array and have them all be a similar velocity and position (where they entered) and to steer a Boid away from another if it gets too close. Then outside the loop have them the entire flock have an average direction the using average velocity of other boids and move same position using average position of other boids.	2

Work Completed:

Features and Tasks	Time Estimate	Time Actual	Complete
Feature 1: Game States	1	1.5	Y
Task 1: Research for implementing game states (Lose, Win, Pause) in Godot.	20 mins	30 mins	Y
Task 2: Creating state machine for each state the game will be in.	10 mins	0	N
Task 3: UI for Pause state with different menu buttons (visual only).	30 mins	1	Y
Feature 2: Loading Screen	1	1	Y
Task 1: Scene that contains a sprite that covers the screens size and is pure black. Create an animation that turns the	30 mins	30 mins	Y

transparency of the sprites alpha to 0. Another animation that turns the sprites alpha from 0 back to 255. (Fade in and Fade Out animations).			
Task 2: Loading script that has access to the different games states in the game manager. Also can disable the Players input during the loading duration.	30 mins	30 mins	Y
Feature 3: Main Menu / Pause Menu	1.5	3	Y
Task 1: Main Menu system for starting the game, exiting the game and buttons for the user to click on.	30 mins	1	Y
Task 2: Visual scrolling back sprite, audio for button clicks aswell as background music.	20 mins	1	Y
Task 3: Pause Menu thats stops gameplay on a button press and resumes gameplay with the same button.	10 mins	30 mins	Y
Task 4: Has three different buttons: Continue, Options, Exit. Options lets you change the audio mixers (SFX, Music). Exit loads the Main Menu scene and exits from gameplay.	30 mins	30 mins	Y
Feature 4: Shop Scene	3	3	Y
Task 1: Shop scene that contains an NPC to interact with that displays a text label with dialogue.	1	1	Y
Task 2: Purchasable entity (Item) that can be interacted with based on an Area2D trigger collider. Interaction will remove the selected amount from Players current currency amount in Player_Data. This removes the static body that is blocking the Player from collecting the item.	1	1	Y

Task 3: Shop Manager that holds the values for each Item and how many Items are available to purchase. Labels that display for each Item with the amount of currency needed to purchase that Item.	1	1	Y
Feature 5: Seal Scene	2	2	Y
Task 1: Seal end scene creation with tile map, camera limits and boundary collision.	30 mins	30 mins	Y
Task 2: Seal entity with sprite, animation, Area2D hitbox with sphere hitbox collider. Code for when dealing damage to Seal when colliders with an Enemy.	1	1	Y
Task 3: Game over state if Seals health is zero. Game win state if Enemy count is zero.	30 mins	30 mins	Y
Feature 6: Seal Attack Enemy (Simple)	3	2	Y
Task 1: Enemy that spawns from offscreen that moves towards the Seals current position in the scene. Has a randomised speed within a range of 2 - 5 pixels per frame.	1	30 mins	Y
Task 2: Enemy that spawns from offscreen but movement is based on a spline. The end node for the spline always leads to the centre of the screen where the Seals position is.	1	20 mins	Y
Task 3: Creation for different splines with different patterns and shapes for balance throughout the end scene for defending the Seal.	1	1	Y
Feature 7: Level Design	1	4	Y

Task 1: Tutorial layout planned and drawn on paper to showcase each feature in the game.	30 mins	1	Y
Task 2: Rough blockout for each scene in Godot using a simple grid tile map.	30 mins	3	Y
Feature 8: Audio Mixers/Option Sliders	1	1	Y
Task 1: Different Mixer Buses for Master, SFX and Music. Import and assign different audio files for the different buses.	30 mins	30 mins	Y
Task 2: UI sliders for changing the volume for the different buses in the Options Menu.	30 mins	30 mins	Y
Feature 9: Audio SFX and Music	1	1	Y
Task 1: Created AudioStream2D for playing music in each scene. Will need to research if needs to be a singleton to play while scene transition.	30 mins	30 mins	Y
Task 2: Created several AudioStream2D for different sounds that play an entities SFX.	30 mins	30 mins	Y
Feature 10: A Star Pathfinding (Simple)	2	5	Y
Task 1: Setup a TileMap Layer that draw different sprites based on a Start and Goal cell locations on the grid	30 mins	2 hours	Y
Task 2: Displays different cells by changing the tile's atlas to a different sprite (Brown to Red) to show the path traveled from the spawn to the target destination.	30 mins	2 hours	Y
Task 3: Pass a reference for the Player and convert it's X/Y coordinates from Vector2	1 hour	1 hour	Y

float into the tile map's int cell format ((0,0) (0,1), (0,2), etc). We do this by using test_a_star.local_to_map(player.global_position)			
Feature 11: Ranged Enemy	1	3	Y
Task 1: Facing direction for firing projectiles (can be assigned in the editor when placing the enemy), UP/DOWN/LEFT/RIGHT.	20 mins	30 mins	Y
Task 2: Instancing a projectile that is inserted into the current scene tree and travels along a set direction. Can damage the player based on tag detection, and then destroy itself.	20 mins	1	Y
Task 3: SFX sounds for Shooting and Taking Damage. Health int variable to track how many hitpoints, ShootCooldown variable to allow for multiple Ranged Enemys to shoot at different intervals and Set Death(), TakeDamage(), knockback() functions.	20 mins	1	Y
Feature 12: Particles (Hit/Death/Pickup)	1	1	Y
Task 1: Animated Sprite that plays a oneshot animated spritesheet. This is a generic particle effect that can be reused.	20 mins	20 mins	Y
Task 2: Instance the particle inside different entities (Player, Enemy, Pickups, etc), This adds it to the parent node tree (scene), then when animation is finished it is queue freed.	20 mins	20 mins	Y
Task 3: Create different particles for different effects (Hit, Death, Pickup).	20 mins	20 mins	Y

Feature 13: Sound Manager	2	2	Y
Task 1: Autoloaded audio streamer that contains every sound file in the game.	20 mins	20 mins	Y
Task 2: Mute functionality to turn off either the Music or SFX mixer buses.	20 mins	20 mins	Y
Task 3: Audio restructuring - remove previous audio streamers from entities and call a "play_X_sound()" function from autoloaded SoundManager instead.	1	1	Y
Feature 14: Destructible Entity (Wall)	1	1.5	Y
Task 1: Area2D with a rectangle collider that listens for a tag with "Projectile". StaticBody2D that blocks Player movement, and is assigned collision layer 2 (Floor/Wall).	20 mins	20 mins	Y
Task 2: On a collision with Projectile, a timer (0.5 seconds) will start that will queue_free the entity. While that happens a SFX sound plays and a particle effect is added to the scene tree.	20 mins	20 mins	Y
Task 3: Disable StaticBody2D and the timer will have ended deleting the entity allowing the Player to pass through. Added a flash effect using a similar shader to the Player's flash.	20 mins	50 mins	Y
Feature 15: Switch (Shoot)	1	1	Y
Task 1: Area2D with a rectangle collider that listens for a tag with "Projectile". Sprite2D for representing the target you shoot. Custom colours created [Idle = (0,0,0), activated = (1,0,0)]. Bool for when the Switch has become active., set to false in Ready() function.	20 mins	20 mins	Y

Task 2: On a collision with Projectile, a timer (0.5 seconds) will start that will queue_free the entity. While that happens a SFX sound plays and a particle effect is added to the scene tree. Active colour assigned when bool is set to true.	20 mins	20 mins	Y
Task 3: Switch emits a signal that is listened for by another entity in the scene tree (Door, Gate, Bridge, etc).	20 mins	20 mins	Y
Feature 16: Switch (Ground)	1	1	Y
Task 1: Area2D with a rectangle collider that listens for a tag with "Player". Sprite2D for representing the target you shoot. Custom colours created [Idle = (0,0,0), activated = (1,0,0)]. Bool for when the Switch has become active., set to false in Ready() function.	20 mins	20 mins	Y
Task 2: On a collision with Projectile, a timer (0.5 seconds) will start that will queue_free the entity. While that happens a SFX sound plays. Active colour assigned when bool is set to true.	20 mins	20 mins	Y
Task 3: An additional check for if the chest requires a key to open, depending on a bool 'b_requires_key' which can be set in the editor view. As well as a movement function that moves the chest back and forth on the X axis before it being deleted (polish).	20 mins	20 mins	Y
Feature 17: Chest (Interact)	1	2	Y
Task 1: Area2D that listens for "Player" tag. When inside a bool 'b_is_active' is set to true and a label displaying a hint text is set to visible. When exited the reverse happens.	20 mins	30 mins	Y

Task 2: Input function for detecting if the “Interact” key is pressed when inside the trigger area. If it is then a bool ‘b_is_opened’ is set to true and starts a timer (0.5 seconds) that will call the spawnItems() function. An animation for the chest plays during this time (3 frames lasts for 2 seconds).	20 mins	1 hour	Y
Task 3: An additional check for if the chest requires a key to open, depending on a bool ‘b_requires_key’ which can be set in the editor view. As well as a movement function that moves the chest back and forth on the X axis before it being deleted (polish).	20 mins	30 mins	Y
Feature 18: Chest (Item Spawn)	1	2	Y
Task 1: Preload different item entities that are randomly selected. Create a separate function for each item so that it can be instanced without any errors.	30 mins	30 mins	Y
Task 2: Items are selected based on how many items a chest can hold (3, 2, 1). Each of these slots then can contain a single item that will be set to spawn at a random position above the chest (Left, Middle, Right). These positions are Node2D in the prefab’s scene.	30 mins	30 mins	Y
Feature 19: Game Over Screen	1	1	Y
Task 1: Created new scene that contains a simple button to return to the Main Menu scene, so that it will create a new instance of the game and remove all progress made (currency, health amount, score, etc).	20 mins	20 mins	Y
Task 2: Label that displays “Game Over”, SFX sound plays to indicate the new state	20 mins	20 mins	Y

and a simple background colour rect that shows red to show the Player has failed.			
Task 3: Game Win Scene is a duplicate of the Game Over scene expect it displays a different label text for “Game Win”, a different SFX sound plays and the colour rect is now green to show the Player has won.	20 mins	20 mins	Y
Feature 19: Boids Flocking algorithm	6	8	Y
Task 1: Simple character Area2D entity that contains a collider for detecting other entities also tagged as “Boid”. A simple sprite to show the Enemy, aswell as a triangular sprite to indicate the direction the entity is facing (faces left by default).	1	1	Y
Task 2: Create several raycasts that allow the Boid to detect collisions in front of itself. 4 raycasts in total (top, middle upper, middle lower, bottom). This gives the Boid a good vision cone to see in front and above itself. Put these raycasts into an array then loop threw them to see if they have collided with any object tagged as “Wall”. If it hits a wall we move in the opposite direction (full 180 degrees) based on the raycast.	1	1	Y
Task 3: Area2D trigger collider that detects other colliders tagged as Boid. Is large (50 X 50 pixels) so that it can detect another Boid easily and prevents clipping with another one. When a Boid enters it is appended into an array and this is used in a function called boids().	2	3	Y
Task 4: Loop through each Boid inside of the array and have them all be a similar velocity and position (where they entered) and to steer a Boid away from another if it gets too close. Then outside the loop have them the	2	3	Y

entire flock have an average direction the using average velocity of other boids and move same position using average position of other boids.			
Bug: Boids have no velocity on spawn due to not being close enough to detect another Boid in it's trigger collider.			
Total Time:	46 hours		

Sprint 2 Video link:
[Sprint_2_Video.mp4](#)

Ranged enemy knockback (GDD), Seal Attacker Spline (redo task), Boids algorithm.

Sprint 3 Report

Features and Tasks	Time (hours)
Feature 1: A* pathing fix	3
Task 1: Creating a Tile Map Layer that reads from a custom made A* grid. The grid is made in a Pathfinding Manager entity that is included in the scene and reference. The parameters of the algorithm are defined here such as the heuristic (Manhattan Distance).	20 mins
Task 2: The tiles are read in as a 1D array (optimisation) and are then sent to the Seal NPC entity for which path is to be taken. The path first starts at the current tile position of the Seal, and the end is the current tile the Player is on. The array then takes the points from the start to end and every point in between depending on obstacles.	20 mins

Task 3: A custom data layer is created so that each tile depending on it's position in the sprite atlas will have different costs in term of weight when generating a path.	30 mins
Feature 2: Hit-stop effect (polish)	1
Task 1: Hit-stop research using youtube, godot forums, articles, etc.	20 mins
Task 2: We freeze the game by setting the actual engine's process itself to 0 with Engine.time_scale = 0. Then inside our function we create temporary timer that ignores timescale and takes in a float we define that is the timer's length (short, medium, long - see GDD for time amounts). When the timer finishes we then reset the time_scale to 1.0	30 mins
Task 3: We make this Hitstop effect a singleton that can be used in any scene and edit every entity that uses it to call this function.	10 mins
Feature 3: Final art implemented	1
Task 1: Importing, resizing and applying nearest neighbour filtering to newest versions for the different spritesheets.	40 mins
Task 2: Modified existing art in photoshop to be unique colours to be used for different enemies (Nagivation Enemy, Seal Attack Enemy, Spline Enemy).	10 mins
Task 3: Created more animations for different effects such as the the Fade effect that is used when transitioning between scenes.	10 mins
Feature 4: Godot Shader Research	1
Task 1: Use tutorials on youtube, articles and forums to gain an understanding of how shaders work in Godot.	30 mins
Task 2: Experimenting with different shaders to create different effects whether screen based or per object (particles).	30 mins

Feature 5: Godot Blur Shader (Pause Screen)	1
Task 1: Create a canvas layer that covers the entire screen size that sits beneath the UI text boxes. This means the UI itself will not get blurred, only the paused gameplay screen below it.	20 mins
Task 2: Expose certain parameters to the inspector tab in the editor so we can change the settings easily and how strong the blur amount is. For this we use a sampler2D which is a texture (entire screen is grabbed as a texture).	20 mins
Task 3: We make a variable called "lod" and set it to 0.0f, this will be used to set our texture sample to a custom Level Of Detail. When we set it to 0.0f we are setting it to the lowest meaning it will blur the most. We use this as an argument in a built-in function called "texelod()". We return this as a vector4 colour inside the fragment shader.	20 mins
Feature 6: Outline Shader	1
Task 1: We create two variables, the first for the outline's colour which is a vector4 (R,G,B,A) and the second is the outline's thickness (float). In the fragment shader we convert our line thickness in pixels into UV coordinates.	20 mins
Task 2: For the outline to work we sample the same texture multiple times in the same direction outwards. We sample the fragment at a certain offset then do it again but in a certain direction, however sampling for the offset for UV coordinates if we want to move left we must go right, meaning directions are reversed.	30 mins
Task 3: We can then assign a colour and line thickness for the shader in the editor since we exposed the parameters with the uniform keyword. We then put this shader into the different entities that require them (Sign when the Player is within it's dialogue interact trigger).	10 mins
Feature 7: Signs	1

Task 1: A simple trigger collider that when the Player is inside and presses the Interact key will display a text box onscreen. The text is loaded in as defined single strings that are separated in JSON format. Pressing interact will load the next sentence until it reaches the end and the dialogue box will then close.	20 mins
Task 2: If there are still sentences inside the JSON file we then call the load_dialogue() function, that opens a specific file we want (we set it as a string path in the editor). We grab the text using "JSON.parse_string(file.get_as_text())" and put it into a variable and return it from the function.	20 mins
Task 3: Each sentence is then set as an ID that we can then use simple addition to go through them in the next_script() function. In this function we can directly access the text for the dialogue box onscreen by setting the string as the returned variable from the load_dialogue() function.	20 mins
Feature 8: Score System	1
Task 1: Simple int variable that's held in Player_Data so it can be reference by other entities easily due to it being a singleton. Add functionality to every entity that can add or decrease the Player's score (Enemy death, Item picked up, Seal damaged, Player Damaged).	30 mins
Task 2: Score display that is shown on the Game Over and Game Win scenes in a text box. This score is then reset on game start.	30 mins

Features and Tasks	Time Estimate	Time Actual	Complete
Feature 1: A* pathing fix	1	5	Y
Task 1: Creating a Tile Map Layer that reads from a custom made A* grid. The grid is	20 mins	2	Y

made in a Pathfinding Manager entity that is included in the scene and reference. The parameters of the algorithm are defined here such as the heuristic (Manhattan Distance).			
Task 2: The tiles are read in as a 1D array (optimisation) and are then sent to the Seal NPC entity for which path is to be taken. The path first starts at the current tile position of the Seal, and the end is the current tile the Player is on. The array then takes the points from the start to end and every point in between depending on obstacles.	20 mins	2	Y
Task 3: A custom data layer is created so that each tile depending on it's position in the sprite atlas will have different costs in term of weight when generating a path.	20 mins	1	Y
Feature 2: Hit-stop effect (polish)	1	1	Y
Task 1: Hit-stop research using youtube, godot forums, articles, etc.	20 mins	20 mins	Y
Task 2: We freeze the game by setting the actual engine's process itself to 0 with <code>Engine.time_scale = 0</code> . Then inside our function we create temporary timer that ignores timescale and takes in a float we define that is the timer's length (short, medium, long - see GDD for time amounts). When the timer finishes we then reset the <code>time_scale</code> to 1.0	30 mins	30 mins	Y
Task 3: We make this Hitstop effect a singleton that can be used in any scene and edit every entity that uses it to call this function.	10 mins	10 mins	Y

Feature 3: Final art implemented	1	2	Y
Task 1: Importing, resizing and applying nearest neighbour filtering to newest versions for the different spritesheets.	40 mins	1 hr 40 mins	Y
Task 2: Modified existing art in photoshop to be unique colours to be used for different enemies (Nagivation Enemy, Seal Attack Enemy, Spline Enemy).	10 mins	10 mins	Y
Task 3: Created more animations for different effects such as the the Fade effect that is used when transitioning between scenes.	10 mins	10 mins	Y
Feature 4: Godot Shader Research	1	1	Y
Task 1: Use tutorials on youtube, articles and forums to gain an understanding of how shaders work in Godot.	30 mins	30 mins	Y
Task 2: Experimenting with different shaders to create different effects whether screen based or per object (particles).	30 mins	30 mins	Y
Feature 5: Godot Blur Shader (Pause Screen)	1	1	Y
Task 1: Create a canvas layer that covers the entire screen size that sits beneath the UI text boxes. This means the UI itself will not get blurred, only the paused gameplay screen below it.	20 mins	20 mins	Y
Task 2: Expose certain parameters to the inspector tab in the editor so we can change the settings easily and how strong the blur amount is. For this we use a sampler2D which is a texture (entire screen is grabbed as a texture).	20 mins	20 mins	Y

Task 3: We make a variable called “lod” and set it to 0.0f, this will be used to set our texture sample to a custom Level Of Detail. When we set it to 0.0f we are setting it to the lowest meaning it will blur the most. We use this as an argument in a built-in function called “texelod(). We return this as a vector4 colour inside the fragment shader.	20 mins	20 mins	Y
Feature 6: Outline Shader	1	1	Y
Task 1: We create two variables, the first for the outline’s colour which is a vector4 (R,G,B,A) and the second is the outline’s thickness (float). In the fragment shader we convert our line thickness in pixels into UV coordinates.	20 mins	20 mins	Y
Task 2: For the outline to work we sample the same texture multiple times in the same direction outwards. We sample the fragment at a certain offset then do it again but in a certain direction, however sampling for the offset for UV coordinates if we want to move left we must go right, meaning directions are reversed.	30 mins	30 mins	Y
Task 3: We can then assign a colour and line thickness for the shader in the editor since we exposed the parameters with the uniform keyword. We then put this shader into the different entities that require them (Sign when the Player is within it’s dialogue interact trigger).	10 mins	10 mins	Y
Feature 7: Signs	1	1	Y
Task 1: A simple trigger collider that when the Player is inside and presses the Interact key will display a text box onscreen. The text is loaded in as defined single strings that are	20 mins	20 mins	Y

separated in JSON format. Pressing interact with load the next sentence until it reaches the end and the dialogue box will then close.			
Task 2: If there are still sentences inside the JSON file we then call the load_dialogue() function, that opens a specific file we want (we set it as a string path in the editor). We grab the text using "JSON.parse_string(file.get_as_text())" and put it into a variable and return it from the function.	20 mins	20 mins	Y
Task 3: Each sentence is then set as an ID that we can then use simple addition to go through them in the next_script() function. In this function we can directly access the text for the dialogue box onscreen by setting the string as the returned variable from the load_dialogue() function.	20 mins	20 mins	Y
Feature 8: Score System	1	1	Y
Task 1: Simple int variable that's held in Player_Data so it can be reference by other entities easily due to it being a singleton. Add functionality to every entity that can add or decrease the Player's score (Enemy death, Item picked up, Seal damaged, Player Damaged).	30 mins	30 mins	Y
Task 2: Score display that is shown on the Game Over and Game Win scenes in a text box. This score is then reset on game start.	30 mins	30 mins	Y

Sprint 3 Video link:
[Sprint 3 Video.mp4](#)

5. Risks and Challenges

Certain design features may cause issues during development and should be addressed during each Sprint Report. These issues may appear during the creation of a feature not fitting the overall balance, the difficulty of programming a feature, or negative results from user testing.

Bugs and Dead-ends

Group Tags:

I encountered issues with triggering different layer collisions/masks. This was due to trying to reuse the same Area2D/Collision Shape2D that is used for physics collision with other level entities (Walls, Moving Obstacles, Hazards), which caused confusion with them being emitted as similar triggers to Damage or Hurt collisions. This was because they were sharing the same Group Tags. The solution was to have different Area2D nodes with unique Group Tags attached rather than them all being assigned the same tags.

Input Handling:

Certain player actions were restricted or not registered by the game due to the inputs being handled entirely within different states.

Audio:

For AudioStreamer2D that are played when an entity is destroyed, the object is freed from memory before it has a chance to fully play the entire length of the sound. So I had to go back to a lot of code and add an await function for each sound, so that it waits for the sound to finish playing before executing the next line of code.

Vignette Shader:

The Vignette Shader can't be used in any scene that contains another Canvas Layer since it then takes priority over mouse inputs for button presses. This broke all menu scenes and was not used for these scenes. If I had more time it would be possible to combine these canvas layers and fix this issue.

6. Feature Design

Room structure and transitioning: Each level is comprised of several small rooms that make up the layouts for the level as a whole. To go between each room we used trigger colliders called "Doors" (doesn't have to visually be a door). Each Door will inherit from a Door interface that requires a reference to a Spawn Object located elsewhere in the level. This allows us to plug in this Spawn Object as a reference so when we transition from one room to the next it will move the Player to that X,Y position. Spawn Objects are simple Nodes that contain an Vector2D and a unique name.

Enemy AI:

Many enemies will follow a simple pattern to both navigate the level as well as during combat with the player. Enemy movement can be limited to within a nav mesh (combat based) or follow along a spline (puzzle based). This is for level design, consistency and balance.

'Slime Enemy' moves in a random pattern within a navmesh zone when in their Idle State. A circle collider trigger acts as their 'Alert Trigger'

7. Art/ Audio/ Animation/ Sound/ UI

Each game world sprite is limited to 16 X 16 pixels. This includes the Player, Enemies, and world tiles (Ground, Walls, Traps, etc). This allows a static screen to hold 10 sprites vertically and 16 sprites horizontally.

Screen dimensions will be dynamic to be resized to a person's monitor size while in fullscreen. The game will still always try to fit in the resolution scaling of 1920 X 1080, 960 X 540, 480 X 270. This is important for playing the game inside the itch.io browser window page, since we can rescale it and keep the aspect ratio. The game can be rescaled in the Options Menu in the Main Menu.

Animation frames will be limited to a low number (4, 6, 8), this way we reduce the amount of work needed for each animation for each entity. We will only need a single frame when exporting from an editor like Aesprite, even if the artist creates the animations on twos (same frame played twice for playback when creating animation)

Shaders will be used for graphical effects for level load transitions, UI elements, Distortion effect for simulaint oil, and certain post processing effects (filters, vignette, CRT scanlines, etc)

8. Summary of Planned Work

For the first sprint, a basic iteration of the core mechanics will be created and tested to guarantee the game structure, balance and overall design.

Certain art, animation and sounds to be created by the artist (Micheal) that can be used for the final showcase (Player, Enemy 1, World art, UI, etc).

The creation of a prototype that can be play tested by users for feedback based on control scheme, intuitiveness of the mechanics, tutorial showcase and how well a user learns.

9. Videos

To be completed after first working build with primary mechanics implemented.

Links: ...

10. Testing

A selection of different users with different skill levels will participate in certain milestone builds of the game to keep track of how easy the game is to play and understand how to play. These sessions will be screen recorded and notes will be made reporting what issues will come up during the playtest. The video recording can be used for replicating bugs, seeing what issues were missed during the playtest aswell as documentation for a list of issues to be fixed during the next milestone or Sprint.

11. Audio List

Music

1. Music_Gameplay
2. Music_Menu

SFX

Common

1. Projectile_Destroyed

Enemy

1. Boss_appear
2. Enemy_Damage
3. Enemy_death
4. Enemy_Shoot
5. Enemy Absorb

Interactables

1. Damage_generic
2. Locked_Door
3. SecretFound
4. Unlock_Door
5. Destructable Wall
6. Secret Found

NPC

1. Seal_Damage
2. Seal_Death

Pickup

1. Pickup_generic
2. Pickup_bonus
3. Pickup_gem_2
4. Pickup_gem_3
5. Pickup_health
6. Pickup_health_full

Player

1. Player_death
2. Player_dodge
3. Player_shoot
4. Player_suck

States

1. Game_Over
2. Game_Win

UI

1. UI_Back
2. UI_Confirm
3. UI_Pause

Shop

1. Purchase_Confirm
2. Purchase_Cancel